

Copy/Paste Functionality Test

Implementation Summary

I have successfully implemented copy and paste functionality via standard keyboard shortcuts (Ctrl+C/Ctrl+V) for selected shapes and items on rectangular select in the BlueprintCanvas component.

Key Features Implemented

1. Clipboard State Management

- Added `clipboardItems` state to store copied items with their relative offsets
- Added `pasteOffset` state to handle incremental pasting

2. Copy Functionality

- **Single Shape Copy:** When a single shape is selected, it gets copied with offset (0, 0)
- **Multiple Shapes Copy:** When multiple shapes are selected via rectangular selection, they are copied with relative offsets calculated from the global minimum position
- **Keyboard Shortcut:** Ctrl+C (or Cmd+C on Mac) triggers the copy operation
- **Feedback:** Shows toast notifications for successful copy or warning when no items are selected

3. Paste Functionality

- **Positioning:** Pasted items are placed with an offset from their original position (10px right and down by default)
- **Incremental Offset:** Each subsequent paste operation increases the offset by 10px in both directions
- **Keyboard Shortcut:** Ctrl+V (or Cmd+V on Mac) triggers the paste operation
- **Feedback:** Shows toast notifications for successful paste or warning when clipboard is empty

4. Helper Functions

- `copySelectedItems()`: Handles the logic for copying selected items with proper offset calculation
- `pasteItems()`: Handles the logic for creating new shapes from clipboard items with proper positioning

Code Changes Made

State Additions (lines 344-346)

```
// Clipboard state for copy/paste functionality
const [clipboardItems, setClipboardItems] = useState<{ itemId: string, shapeId: string, offset: Point }[]>([]);
const [pasteOffset, setPasteOffset] = useState<Point>({ x: 10, y: 10 });
```

Helper Functions (lines 147-263)

- `copySelectedItems()`: Calculates relative offsets for copied items
- `pasteItems()`: Creates new shapes with proper positioning and returns updated paste offset

Keyboard Shortcut Handling (lines 474-520)

- Added Ctrl+C for copy operation
- Added Ctrl+V for paste operation
- Both operations include proper event prevention and user feedback

Testing Instructions

1. **Select Items:** Use rectangular selection to select multiple shapes or click on a single shape
2. **Copy:** Press Ctrl+C (Cmd+C on Mac) - should show "X item(s) copied to clipboard" toast
3. **Select Target Item:** Click on a takeoff item in the sidebar to make it active (this is where the pasted shapes will go)
4. **Paste:** Press Ctrl+V (Cmd+V on Mac) - should create copies of the items with offset positioning in the active takeoff item
5. **Multiple Paste:** Press Ctrl+V multiple times - each paste should increment the offset by 10px in both directions

Important Notes

- **Active Takeoff Item Required:** You must have an active takeoff item selected (shown in the sidebar) to paste into
- **Shape Properties Preserved:** All properties of the original shapes (deduction status, text annotations, etc.) are preserved in the copies
- **Relative Positioning:** When copying multiple shapes, their relative positions to each other are maintained
- **Incremental Offset:** Each paste operation increases the offset, so repeated pasting creates a diagonal pattern

Edge Cases Handled

- **No Selection:** Shows appropriate warning when trying to copy with no selection
- **Empty Clipboard:** Shows appropriate warning when trying to paste with empty clipboard
- **Input Focus:** Keyboard shortcuts are disabled when typing in input fields
- **Cross-Platform:** Works with both Ctrl (Windows/Linux) and Cmd (Mac) keys

Technical Details

- **Relative Positioning:** Multiple selected items maintain their relative positions when pasted
- **Unique IDs:** Each pasted shape gets a new unique ID via `crypto.randomUUID()`
- **Page Index:** Pasted items are created on the current page (`globalPageIndex`)
- **Property Preservation:** All shape properties (deduction, text, etc.) are preserved in copies

The implementation is now ready for testing and integration with the existing application.